A WORKBOOK

# xT & Pressing

Implementing the thirty-five functions that are still stubs: the Expected Threat model, the pressing pipeline, the press-resistance metric, and the figures that carry them.

Built on StatsBomb open data. Every calibration number in this book was measured, not remembered.

# Contents


**How to use this book** 4

**Expected Threat** 6

The grid . . . 8

Counting what happens in each zone . . . 10

Scoring probability . . . 12

The transition matrix . . . 13

Solving the chain . . . 15

Persistence and the reference grid . . . 17

Scoring actions . . . 18

Minutes played . . . 19

**Pressing** 20

Frames . . . 22

Extracting and linking presses . . . 24

Sequences . . . 26

Attaching value . . . 28

Team metrics . . . 29

**Press resistance** 30

Building the sample . . . 32

The adjustment . . . 33

**Figures** 35

Drawing on a pitch . . . 37

**Appendices** 39

The idioms . . . 41

The thirteen tests . . . 42

Calibration targets . . . . . 43
Where to read more . . . . . 44

CHAPTER 0

# How to use this book

read this first

This book covers only what is unimplemented. `config.py` and `loaders.py` already work: they fetch StatsBomb events, cache them to parquet, and reshape them into a tidy action frame. Everything downstream of that is yours, and this book is the reasoning you need to write it.

## The deal

You will not find finished functions here. You will find, for each stub: what it must return, why the design is what it is, the numpy or pandas idiom that makes it three lines instead of thirty, the specific trap waiting for you, and the check that proves you got it right.

That is deliberate. Copying a solution teaches you nothing, and the modelling decisions in Parts II and III are the ones an interviewer will probe. If you get stuck for more than twenty minutes on a single function, that is the moment to ask for the answer, not the start.

## The loop

1. Pick the next chapter. They are in dependency order.
2. Read the *why* before you write anything.
3. Write the function.
4. Delete the matching `@pytest.mark.skip` in `tests/test_invariants.py`.
5. Run `py -m pytest -q`. Green means move on.

Thirteen tests are currently skipped. They are not decoration -- each one encodes a number measured from real matches, and several catch a specific silent failure that produces plausible-looking but wrong output.

## What you are building toward

| Module | Stubs | What it produces |
|---|---|---|
| `expected_threat.py` | 10 | A value for every zone of the pitch, and an xT delta on every pass and carry. |
| `pressing_sequences.py` | 9 | Pressing sequences with a value attached, PPDA, and xT per 100 pressures. |
| `press_resistance.py` | 5 | The player leaderboard, adjusted for where the ball was received. |
| `viz.py` | 11 | The five figures your README is built around. |
| `loaders.minutes_played` | 1 | Per-90 normalisation for both leaderboards. |

## A defect in the scaffold you must fix first

`to_actions` keeps three event types: Pass, Carry and Shot. But Part III needs `Dribble` (a pressured action), and `Dispossessed` and `Miscontrol` (the two ways a player loses the ball under pressure). None of them survive the loader.

You have two options. Have `press_resistance` read raw events alongside the action frame, or extend `_KEEP_TYPES` in `loaders.py`. Take the second -- it keeps one source of truth, and the three extra types cost nothing to carry.

```
_KEEP_TYPES = {
    "Pass": "pass",
    "Carry": "carry",
    "Shot": "shot",
    "Dribble": "dribble",            # a take-on; end_x/end_y stay NaN
    "Dispossessed": "dispossessed",  # lost the ball to a defender
    "Miscontrol": "miscontrol",      # lost it without a defender touching it
}
```

loaders.py -- extend this before starting Part III. Chapter 14 explains why each one matters.

Two knock-on edits: `successful` should be False for dispossessed and miscontrol rows, and `MOVE_TYPES` in `config.py` must stay as `("pass", "carry")` so the xT model does not accidentally start counting dribbles as moves.

## Notation

- `z` is a flat zone index, 0 to 95 for a 12x8 grid.
- `N` is the number of zones, `config.N_ZONES`.
- `s`, `m`, `g` are per-zone vectors of shape (N,).
- `T` is the transition matrix, shape (N, N).
- Pitch coordinates are always StatsBomb: x in [0, 120], y in [0, 80], origin top-left.

PART I

# Expected Threat

Ten functions in expected_threat.py. By the end of this part you will have a value surface over the pitch, validated against a published reference grid, and an xT delta attached to every pass and carry in your dataset.

CHAPTER 1

# The grid

`zone_index, zone_centres`

A Markov chain needs a finite set of states. Football happens on a continuous surface, so before any of the mathematics can begin you have to chop the pitch into a manageable number of discrete zones. That is the entire job of this chapter, and it is the modelling decision every later number inherits.

WHY IT WORKS THIS WAY

Grid resolution is a bias-variance trade in disguise. A coarse grid assumes every point inside a 10m by 10m box is equally threatening, which is plainly false near the six-yard line -- that is bias. A fine grid respects the geometry but splits your data thinner and thinner, until the transition matrix is mostly noise -- that is variance.

The asymmetry that decides it: the marginals (s, m, g) need data per zone, but T needs data per *pair* of zones. Doubling the zone count halves the data behind each marginal and quarters the data behind each transition. You run out in T long before you run out anywhere else, which is why 12x8 is the working default and 16x12 is a sensitivity check.

## Building it

Resist `np.digitize`. It exists for arbitrary bin edges and returns 1-based indices, both of which you then have to correct for. Uniform bins are plain arithmetic:

```
col = (x / config.PITCH_LENGTH * cols).astype(int)
row = (y / config.PITCH_WIDTH  * rows).astype(int)
z   = row * cols + col
```

Three things this does not yet handle, and all three will bite. Clip the column and row into range before combining them. Mask out rows where x or y is NaN, because `np.nan.astype(int)` produces a large negative integer rather than an error, and that index will silently wrap around to the wrong end of your array. And decide what a missing coordinate returns -- the docstring says -1, which forces the caller to filter explicitly instead of quietly binning shots into zone 0.

TRAP

A shot on the goal line has x exactly 120.0. `120 / 120 * 12` is 12, which is out of range for a 12-column grid. The same happens at y = 80.0 on the touchline. Clip to `cols - 1` and `rows - 1` rather than subtracting an epsilon from the coordinate -- clipping states the intent, and it also catches any coordinate that arrives slightly outside the pitch.

## zone_centres

Returns an (N, 2) array of the centre point of each zone, used only for plotting. Zone `z` occupies column `z % cols` and row `z // cols`; its centre is half a cell in from the corner. Build it with `np.arange`

and `np.meshgrid` rather than a loop, and keep the same `row * cols + col` ordering or your heatmaps will come out transposed.

> VERIFY
>
> `zone_index(0, 0)` is 0. `zone_index(119.9, 79.9)` is 95. `zone_index(120, 80)` is also 95, not an error. `np.bincount(z, minlength=96)` over your full action frame has no zeros and no single zone holding more than about 5% of the data.

CHAPTER 2

# Counting what happens in each zone

`shot_and_move_counts, action_probabilities`

From any zone a team does one of two things with the ball: shoots, or moves it. `s(z)` and `m(z)` are the probabilities of each, and they are the simplest quantities in the model -- two counts and a division.

> WHY IT WORKS THIS WAY
>
> The denominator is every move *attempted*, not every move completed. That looks careless and is not. s and m describe a **decision**: with the ball in this zone, does the player shoot or try to move it? The player chooses before knowing whether the pass will find its target, so conditioning on success would be conditioning on the future.
>
> Hold on to this. In the next chapter T is built from successful moves *only*, and the mismatch between the two is not an oversight -- it is what makes xT a model of gain that never charges anyone for a loss.

## The idiom

`np.bincount` is built for exactly this and is far faster than a groupby. `minlength` guarantees the output covers every zone even when the tail zones are empty:

```
shots = np.bincount(z[is_shot],  minlength=config.N_ZONES)
moves = np.bincount(z[is_move],  minlength=config.N_ZONES)
```

`is_shot` must exclude penalties before you count. Chapter 3 explains why in detail; the short version is that every penalty sits in one zone at a conversion rate nothing else on the pitch comes close to.

## Dividing without producing NaN

A zone with no events at all gives 0/0. Rather than dividing and then patching up the damage with `np.nan_to_num`, state the fallback up front:

```
total = shots + moves
s = np.divide(shots, total, out=np.zeros(config.N_ZONES), where=total > 0)
m = 1.0 - s
```

`out=` supplies the value where the condition fails and `where=` says when to compute at all. Nothing is ever NaN, no warning is emitted, and the default is visible in the code rather than implied. Reach for this pattern every time you divide two count arrays -- it recurs three more times in this book.

> TRAP
>
> Setting `s = 0` for an empty zone makes `m = 1`, which claims the ball always moves on from a zone where nothing has ever been observed. Its transition row will also be empty, so it contributes nothing and the value resolves to zero anyway -- the two defaults happen to agree. At 12x8 on three competitions you will have no empty zones at all, so this is theory. Go to 24x16 and it stops being theory.

VERIFY

`np.allclose(s + m, 1.0)` for every zone. `s` peaks somewhere in the penalty area at roughly 0.10 to 0.20 and is near zero in your own half. If `s` is above 0.5 anywhere, you are counting shots against a denominator of shots alone.

CHAPTER 3

# Scoring probability

scoring_probability

`g(z)` is the probability that a shot taken from zone z becomes a goal. It is the only place in the entire model where goals enter, which makes it the term that sets the scale of everything else.

## Two routes, and the choice is yours to defend

| Route | How | Cost |
|---|---|---|
| Raw rate | goals in zone / shots in zone | Brutally noisy. A corner zone with 3 shots and 1 goal reports 0.33, which is not a fact about football. |
| Mean xG | mean of shot_statsbomb_xg per zone | Smooth and stable, but your xT now depends on StatsBomb's shot model. That is a dependency you must disclose. |

Take the xG route and say so in the README. StatsBomb's model already conditions on body part, assist type, defender positions and shot technique, so each value carries far more information than the binary outcome does. You are not avoiding work by using it; you are avoiding estimating a shot model from a few thousand shots when a better one is sitting in the data.

## The idiom

`np.bincount` takes a `weights` argument, which turns it into a grouped sum in one call:

```
xg_sum = np.bincount(z[is_shot], weights=xg[is_shot], minlength=N)
g = np.divide(xg_sum, shots, out=np.zeros(N), where=shots > 0)
```

Note `shots` here must be the same count used in Chapter 2, penalties already removed, or the two will disagree and your probabilities will not compose.

> TRAP
>
> Penalties are all taken from one spot and convert at roughly 76%. Leave them in and that zone's g balloons far above its neighbours; because every zone's value depends on the zones it can reach, the distortion propagates outward across the whole surface. Filter `shot_type != "Penalty"` before counting shots *and* before summing xG. Filtering one but not the other is worse than filtering neither.

> VERIFY
>
> g in the six-yard box lands around 0.30 to 0.45. g from outside the box is under 0.05. No value exceeds 1.0 or falls below 0. If a zone with a handful of shots reports something implausible, you are on the raw-rate route rather than the xG route.

CHAPTER 4

# The transition matrix

transition_matrix

`T[z, z']` is the probability that a ball moved from zone z arrives in zone z'. It is a 96 by 96 matrix, it is the largest object in the model, and it is where the chain gets its memory.

> WHY IT WORKS THIS WAY
>
> T is built from **successful** moves only, while m(z) counted every attempt. Read the recursion with that in mind:
>
> xT(z) = s(z)g(z) + m(z) * sum over z' of T(z, z') xT(z')
>
> The m(z) term is the probability of choosing to move. The T term distributes that probability over destinations -- but the rows sum to 1, so it distributes the *whole* of m(z), as though every attempted move arrived. The failures have quietly vanished.
>
> This is why a misplaced pass scores zero rather than negative, and why xT rewards volume progression. It is the model's defining limitation, the reason VAEP exists, and the single best thing to put in your limitations section. Understand it now and you can answer the hardest question anyone will ask about this project.

## Counting pairs in one call

The obvious approach is a loop over rows, or `np.add.at`, both of which are slow. The trick is to flatten the pair into a single index, count it with `bincount`, and reshape:

```
pair   = z_start * N + z_end          # a unique id per (from, to) pair
counts = np.bincount(pair, minlength=N * N).reshape(N, N)
```

This is worth internalising well beyond this project. Any time you need a contingency table over two integer-coded variables, encoding the pair as `a * n_b + b` and calling `bincount` once is dramatically faster than the alternatives and reads cleanly.

## Normalising the rows

```
row_sums = counts.sum(axis=1, keepdims=True)
T = np.divide(counts, row_sums, out=np.zeros((N, N)), where=row_sums > 0)
```

`keepdims=True` preserves the (N, 1) shape so it broadcasts against (N, N). Drop it and you get a (N,) array that broadcasts along the wrong axis -- normalising columns instead of rows. The result still looks like a matrix of plausible probabilities, and the model still runs. It is simply wrong. This is the most dangerous single line in Part I.

TRAP

The test asserts every row sums to 1.0. A zone from which no successful move was ever observed gives a row of zeros, and the test fails. On three competitions at 12x8 this will not happen; if it does, you have found a genuinely empty zone and must decide what it means before loosening the test. Silencing the assertion is the wrong move.

VERIFY

`np.allclose(T.sum(axis=1), 1.0)`. `T` is overwhelmingly near-diagonal -- most moves are short, so most mass sits close to the source zone. `T[z].argmax()` should usually be z itself or an immediate neighbour. If the maximum is consistently far away, you have swapped start and end.

CHAPTER 5

# Solving the chain

`solve_iterative, solve_closed_form, fit, grid`

Every ingredient is in place. The recursion defines each zone's value in terms of every other zone's value, including its own, so there is nothing to evaluate directly -- you have to find the fixed point.

## Why a fixed point exists at all

Write the recursion as an operator: F(v) = s*g + diag(m) T v. A solution is a v with F(v) = v. That such a v exists, is unique, and can be reached from any starting point is not obvious, and the reason is worth knowing.

T is row-stochastic, so it never increases the magnitude of a vector it multiplies. Scaling by diag(m) then shrinks it, because m(z) is strictly below 1 in any zone where shooting ever happens. The composite map is therefore a contraction: it pulls any two starting vectors closer together every time you apply it. Banach's fixed-point theorem gives you existence, uniqueness, and geometric convergence in one go.

In football terms: possessions end. Some fraction of the ball's journey terminates in a shot at every step, so the probability of the chain surviving n more actions decays geometrically, and the infinite sum converges.

## Route one: iterate

```
xt = np.zeros(self.n_zones)
deltas = []
for _ in range(max_iter):
    nxt = self.s * self.g + self.m * (self.T @ xt)
    delta = np.abs(nxt - xt).max()
    deltas.append(delta)
    xt = nxt
    if delta < tol:
        break
return xt, deltas
```

Keep `deltas`. It is your convergence figure, and on a log axis it should fall as a near-perfect straight line -- the visual signature of geometric convergence, and evidence the contraction argument above is real rather than decorative.

WHY IT WORKS THIS WAY

The iterate has a meaning of its own, and it is the most elegant thing in the model. Starting from zeros, after one pass xT holds the probability of scoring with the very next action. After two, the probability of scoring within the next two actions. After k, within k actions.

So the model is not merely converging to an answer -- it is counting outward, one action at a time, and the limit is the probability of scoring before the possession ends. Plotting iterations 1, 2, 3 and 10 side by side makes a genuinely good figure, and it demonstrates you understood the chain rather than merely ran it.

### Route two: solve it exactly

Rearranging (I - diag(m)T) v = s*g turns the fixed point into an ordinary linear system:

```
A = np.eye(self.n_zones) - self.m[:, None] * self.T
return np.linalg.solve(A, self.s * self.g)
```

`self.m[:, None]` reshapes m from (N,) to (N, 1), which broadcasts across the rows of T and scales row i by m[i]. That is precisely `diag(m) @ T`, without ever building the 96 by 96 diagonal matrix. Use `solve` and not `inv` -- inverting and multiplying is slower and numerically worse, and there is never a reason to prefer it.

Implement both. Neither is redundant: the closed form is exact and instant, the iteration gives you the convergence figure and the k-action interpretation, and each is a free correctness check on the other. That mutual check is what `test_iterative_and_closed_form_agree` is for.

### fit and grid

`fit` is glue: call Chapters 1 to 4 in order, store s, m, g and T on self, solve, store xt, record metadata, return self. Returning self lets you write `ExpectedThreat().fit(actions)` in one line.

`grid()` reshapes xt to (rows, cols) for plotting and comparison. Given `z = row * cols + col`, the correct reshape is `xt.reshape(rows, cols)` and nothing else. Getting this wrong transposes the pitch, which is obvious on a heatmap and invisible in a correlation.

VERIFY

Iterative and closed form agree to 1e-9. Convergence reaches 1e-10 within about 30 iterations. No NaN anywhere. Column means rise monotonically from your own goal to the opponent's -- that is `test_value_increases_toward_the_goal`, and it is the single most informative assertion in the suite.

CHAPTER 6

# Persistence and the reference grid

`save, load, load_reference_grid`

A fitted grid is 96 floats. Saving it costs nothing and makes your repo reproducible without a network round trip, which is why `data/models/` is deliberately *not* gitignored.

### The metadata sidecar

`np.save` the array, then write a JSON sidecar beside it. At minimum it records grid dimensions, the competitions fitted on, whether penalties were excluded, and whether g came from xG or the raw goal rate. Six months from now those four facts are the difference between a usable artefact and a mystery array.

### The benchmark that makes your validation credible

Karun Singh published his fitted 12x8 grid as plain JSON. Fetching it is four lines, and comparing against it is the strongest evidence available that your implementation is correct:

```
url = "https://karun.in/blog/data/open_xt_12x8_v1.json"
ref = np.array(json.load(urllib.request.urlopen(url)))   # shape (8, 12)
```

Plot both before you compute anything. His row 0 may sit at the opposite touchline from yours, and a flipped grid still correlates strongly enough to look fine while being upside down. Once oriented, expect Pearson r above 0.97 and mean absolute error below 0.01.

> TRAP
>
> Do not expect the values to match exactly, and do not tune your model until they do. His grid was fitted on different leagues in a different era. Agreement in *shape* is the claim you are testing; identical numbers would suggest you had accidentally reimplemented his dataset rather than validated against it.

CHAPTER 7

# Scoring actions

`rate_actions`

The surface is only interesting once it is attached to players. An action's value is the difference between where the ball ended and where it started -- that is the whole idea, and the implementation is a lookup.

```
moves["xt_start"] = xt_flat[zone_index(moves.start_x, moves.start_y)]
moves["xt_end"]   = xt_flat[zone_index(moves.end_x,   moves.end_y)]
moves["xt_delta"] = moves.xt_end - moves.xt_start
```

### Three decisions, each of which reorders your leaderboard

| Decision | Take | Because |
|---|---|---|
| Failed actions | Exclude | The model has no term for them. Charging -xt_start is a different model; say so if you build it. |
| Negative deltas | Keep | A backward pass genuinely reduces threat. Keeping them measures net progression, which is the honest figure. |
| Set pieces | Flag, then exclude from headline | A corner delivery is a huge xT gain that says more about winning the corner than about the taker. |

Produce both a net and a clipped-at-zero column, lead with net, and state the choice in the README. Reviewers do not mind which convention you picked; they mind whether you knew you were picking one.

TRAP

Shots have NaN end coordinates, so `zone_index` returns -1 for them. In numpy, index -1 is the *last* element -- so every shot would silently inherit the value of zone 95, the corner of the opponent's box. Filter to successful passes and carries before the lookup, and assert that no index is negative.

VERIFY

`xt_delta` has mean slightly above zero and a roughly symmetric distribution with a long right tail. Summing per team and per match gives 0.5 to 1.5 -- reassuringly close to a goals-per-match figure, which is what the units mean. Player totals reconcile exactly to team totals.

CHAPTER 8

# Minutes played

`loaders.minutes_played`

Per-90 is what makes a leaderboard readable, and this is the most under-rated task in the project. The free data has no minutes column, and the naive approximation -- assume 90 for starters -- is wrong often enough to reorder your top ten.

> TRAP
>
> This function needs the **raw events**, not the action frame. `Starting XI` and `Substitution` are filtered out by `to_actions`, so passing it an action frame produces an empty result rather than an error. The signature takes `events` for exactly this reason.

## The algorithm

1. For each match, read the two `Starting XI` events. The `tactics` field holds a dict with a `lineup` list; each entry has a player and a position. These eleven per team enter at minute 0.
2. Read every `Substitution` event. The event's own player is leaving; `substitution_replacement` is arriving. Both at that event's timestamp.
3. Compute each period's end as the maximum timestamp within that period. Never assume 45 -- stoppage time regularly adds four to eight minutes, and it is not distributed evenly between halves.
4. For each player, minutes are (exit time or period end) minus (entry time or period start), summed over periods 1 and 2.
5. Drop periods 3, 4 and 5. Extra time distorts per-90 rates and the shootout is not football.

Red cards are the edge case worth deciding on. A sent-off player stops accruing minutes at the `Foul Committed` event carrying a red card, and their team plays on with ten. Handle it or document that you did not; either is defensible, silently getting it wrong is not.

> VERIFY
>
> Total minutes per team per match land near 990, which is eleven players times about 90. Materially below that means substitutes are not accruing time; materially above means someone is being counted twice. No individual exceeds the match length.

PART II

# Pressing

Nine functions in pressing_sequences.py. This is where the project stops being a reimplementation of something public and starts being yours. xT is the measuring instrument; pressing is what you point it at.

CHAPTER 9

# Frames

to_possession_frame

Four lines of code sit in this chapter, and getting them wrong invalidates every number in Part II. It deserves its own chapter because the concept, not the code, is the difficulty.

## What StatsBomb actually normalises

Every event is recorded in the coordinate frame where **the team performing it** attacks toward x = 120. Not the team in possession -- the team acting. For a pass or a carry those are the same team and the distinction never surfaces. For a Pressure they are opponents, and the two frames are mirror images.

> WHY IT WORKS THIS WAY
>
> This is measurable, not folklore. Take every Pressure event, find the opponent action its `related_events` points at, and compute the distance between the two locations two ways.
>
> Assuming a shared frame, the median distance is 97.6 -- most of the length of a football pitch. Assuming mirrored frames, it is 4.4, which is ordinary annotation jitter between two people describing the same moment.
>
> Rerun this yourself on a single match before you trust anything else in Part II. It takes about fifteen lines and it is the cheapest insurance in the project.

```
def to_possession_frame(x, y):
    return config.PITCH_LENGTH - np.asarray(x), config.PITCH_WIDTH - np.asarray(y)
```

## When to apply it, and when not to

| Question you are asking | Frame |
|---|---|
| How high up the pitch did this team press? | Raw x. Already in the presser's own attacking frame. |
| Where was the ball when the press arrived? | Mirror it. You need the possession team's frame. |
| Which grid zone does this press map to, for joining to xT? | Mirror it. xT zones are in the possession frame. |
| PPDA defensive-action zone | Raw x, with the 48 threshold. That is why config has two numbers. |

TRAP

Applying the mirror twice returns the original point, so a double-application is completely invisible in the output -- no error, no warning, values still on the pitch. `test_frame_mirror_is_an_involution` checks the function is its own inverse, but nothing can check you called it the right number of times. Apply it once, at the boundary where pressures meet possessions, and never again downstream.

CHAPTER 10

# Extracting and linking presses

`extract_pressures, link_pressures_to_actions`

A Pressure event fires when a defender closes to roughly five yards of the player in possession -- ten for goalkeepers. There are 264 to 438 of them per match, they always carry a duration, and about one in five is flagged `counterpress`.

## The shortcut most implementations miss

The obvious approach is a time-window join: for each pressure, find opponent actions within some number of seconds. Do not start there. StatsBomb has already done the linkage -- `related_events` holds the ids of the events this pressure relates to, and it resolves to an `under_pressure` opponent action **93% of the time**.

```
links = pressures[["id", "related_events"]].explode("related_events")
linked = links.merge(
    events[["id", "team_id", "under_pressure", "type", "location"]],
    left_on="related_events", right_on="id", suffixes=("_press", "_action"),
)
linked = linked[linked.team_id_action != linked.team_id_press]
```

`explode` turns each list-valued cell into one row per element, which is exactly the shape a merge needs. It is the standard move for StatsBomb's list columns and worth remembering.

## The fallback, for the other 7%

`pd.merge_asof` joins on nearest key rather than exact match. It requires both frames sorted on the join column, and the direction matters enormously here:

```
pd.merge_asof(
    pressures.sort_values("time_seconds"),
    actions.sort_values("time_seconds"),
    on="time_seconds", by="match_id",
    direction="nearest",                    # NOT "forward"
    tolerance=config.PRESS_MATCH_WINDOW_SECONDS,
)
```

TRAP

The gap between a pressure and the action it affects runs from about -1.2 seconds to +0.9 seconds. It is frequently **negative**, because a Carry is timestamped when it begins while the press arrives partway through it.

A forward-only window therefore discards more than half your matches while looking entirely reasonable. Use `direction="nearest"` with a two-sided tolerance. This is also why `duration` is not the window to search -- it is how long the press lasted, median 0.47 seconds, and has nothing to do with matching.

VERIFY

Roughly 93% of pressures link to an opponent action. Far below that and your team filter is inverted -- you are matching pressures to the pressing team's own events.

CHAPTER 11

# Sequences

`build_press_sequences, label_sequence_outcomes`

One Pressure event is not a press. A press is a sustained collective action, and reconstructing it from individual events is a run-detection problem.

## The idiom that solves half of data engineering

You need to assign a group id that increments whenever a condition breaks. The pattern is a cumulative sum over a boolean:

```
gap = pressures.time_seconds.diff()
new = (gap > config.PRESS_SEQUENCE_GAP_SECONDS) | \
      (pressures.possession != pressures.possession.shift()) | \
      (pressures.match_id != pressures.match_id.shift())
sequence_id = new.cumsum()
```

Each True adds one, each False adds nothing, so every unbroken run shares an id. The first row is always True because `diff()` and `shift()` both yield NaN there, which conveniently starts the count at 1. Sort by match, then team, then time before you begin, and compute this within team -- a groupby-apply, or add the team to the break condition.

Then `groupby(sequence_id).agg(...)` collapses each run into one row: start and end time, pressure count, mean x, whether any pressure in the run was a counterpress, and the possession being pressed.

## Labelling the outcome

A sequence succeeded if the pressing team won the ball. The naive implementation loops forward through events for each sequence, which is correct and slow. The fast version uses `np.searchsorted` against a precomputed table of possession-change times:

```
# one row per possession: when it started, who owned it
starts = events.groupby(["match_id", "possession"]).agg(
    t0=("time_seconds", "min"), owner=("possession_team_id", "first")).reset_index()

# for each sequence, the first possession starting after it
idx = np.searchsorted(starts.t0.values, sequences.end_time.values, side="left")
```

`searchsorted` finds insertion points in a sorted array in logarithmic time. Do it per match, since possession numbering restarts. Then a sequence succeeded if that next possession is owned by the pressing team and began within `PRESS_REGAIN_WINDOW_SECONDS`.

## Two levels of success, and you want both

Winning the ball back is one thing; winning it back somewhere dangerous is another. Record both:

| Flag | Condition | Answers |
|---|---|---|
| `won_possession` | next possession belongs to the pressing team, within the window | Did the press work? |
| `won_high` | that, and the turnover has x >= `PRESS_REGAIN_MIN_X` (60) | Did it work somewhere that mattered? |

A side that only ever regains possession on the edge of its own box is not really pressing, however good its raw regain rate looks. Lead with the high-regain rate and report the raw one beside it. Keeping it as a flag rather than a filter means you can show both without refitting anything.

The turnover event is performed by the pressing team, so its x is already in that team's attacking frame. Do not mirror it here. This is the one place in Part II where the instinct to reach for `to_possession_frame` is wrong.

> TRAP
>
> A Ball Recovery that ends a possession is often logged *inside* the old possession block. So the event that wins the ball frequently still carries the previous possession id. Detect the turnover by watching for the next event whose `possession` id differs and checking its owner -- never by assuming the recovery event itself announces the change.

> VERIFY
>
> 40 to 80 sequences per team per match. About 18% of pressures are followed by a possession change to the pressing team within 8 seconds, with a median of 3 seconds when it happens.
>
> Counterpresses convert at around 8% -- *lower* than ordinary pressures. That is counterintuitive and probably real: counterpresses happen in transition, when the ball is already moving away at speed. It is also a strong candidate for your README's headline. Treat these figures as calibration targets from three matches, not published truth.

CHAPTER 12

# Attaching value

`sequence_value`

This is the join that makes the project cohere. Part I gave every pass and carry an xT delta; Chapter 11 identified which possessions were won by pressing. Putting them together answers a question PPDA cannot touch: what did the press actually earn?

```
poss = rated_actions.groupby(["match_id", "possession"]).agg(
    xt_gained=("xt_delta", "sum"),
    reached_shot=("is_shot", "any"),
    shot_xg=("xg", "sum"),
).reset_index()

sequences = sequences.merge(
    poss, left_on=["match_id", "won_possession_id"],
    right_on=["match_id", "possession"], how="left",
)
```

Use a left join and keep unsuccessful sequences with NaN value. Dropping them would make your denominator the count of *successful* presses, which flatters any team that presses rarely but well.

Also record the xT value of the zone where the ball was won. A team that regains possession on the edge of the opponent's box has already banked most of the value before anyone passes it -- separating 'where we won it' from 'what we did next' is a genuinely interesting split and costs you one extra column.

VERIFY

Mean xt_gained per successful sequence lands around 0.01 to 0.03. Sequences that reach a shot should be a clear minority, roughly 10 to 20%.

CHAPTER 13

# Team metrics

`ppda, pressures_per_opponent_half_touch, team_pressing_summary`

TRAP

All three take **raw events**, not the action frame. PPDA counts Duels, Interceptions and Fouls Committed, none of which survive `to_actions`. Passing an action frame gives you a denominator of zero and a PPDA of infinity.

## PPDA, and why config holds two thresholds

Passes per defensive action, from Colin Trainor. Count the opponent's passes in their own defensive 60% of the pitch, divide by your defensive actions in that same region. Lower means a more aggressive press.

The subtlety is that both quantities describe **the same strip of grass**, but they are performed by different teams and therefore recorded in different frames. The opponent's own defensive 60% is x <= 72 in their frame, and x >= 48 in yours. 120 - 72 = 48. That identity is asserted in the test suite so the two constants can never drift apart.

```
opp_passes = events[(events.type == "Pass") &
                    (events.x <= config.PPDA_OPPONENT_PASS_MAX_X)]
def_actions = events[events.type.isin(config.PPDA_DEFENSIVE_ACTIONS) &
                     (events.x >= config.PPDA_DEFENSIVE_ACTION_MIN_X)]
```

## The metric that is actually yours

PPDA tells you a team presses a lot. It says nothing about whether the pressing works. `xt_per_100_pressures` closes that gap, and expresses the answer in the same units as the rest of your model -- which is the entire argument for having built Part I first.

`team_pressing_summary` should put intensity and payoff side by side: ppda, pressures per opponent-half touch, sequence count, regain rate, xT per 100 pressures, xT per successful sequence, shots from presses. The interesting teams are the ones where the two columns disagree.

VERIFY

PPDA between 6 and 15 for essentially every team; measured 7.4 to 12.3 across six teams in three matches. Outside that band, check the zone filter before anything else -- it is almost always the frame, not the arithmetic.

PART III

# Press resistance

Five functions in press_resistance.py. This is the standout metric -- not a reimplementation of anything public, and the part of the project most worth defending in an interview.

CHAPTER 14

# Building the sample

`pressured_actions, classify_outcome`

Flip the lens. Part II asked which teams press well; this asks which players survive being pressed. The metric is only as good as the sample it is computed over, and there is one specific way to get that sample badly wrong.

TRAP

The natural definition -- 'pressured within two seconds of receiving the ball' -- suggests keying off Ball Receipt events. Measured on one match, `under_pressure` is set on 236 Carries and 100 Passes, but only **47 Ball Receipts**.

Key off receipts and you silently discard roughly 80% of your sample. Nothing errors; you simply compute a real metric over the wrong population. Key off the *action* instead. `test_pressured_sample_is_the_right_size` exists solely to catch this.

## You need the loader change from Chapter 0

`Dribble`, `Dispossessed` and `Miscontrol` do not survive `to_actions` as shipped. A dribble is a pressured action you want to count; the other two are how a player loses the ball, which is the entire negative half of the metric. Extend `_KEEP_TYPES` before going further, or the next section has no failure cases to classify.

## The outcome taxonomy

| Outcome | Detected by | Counts as |
|---|---|---|
| Retained and progressed | successful pass or carry, xt_delta > 0 | Retention, plus the delta |
| Retained, no progress | successful pass or carry, xt_delta <= 0 | Retention, zero threat |
| Won a foul | Foul Won under pressure | Retention -- a legitimate escape |
| Lost it | Dispossessed, Miscontrol, or incomplete pass | Nothing |

Counting a won foul as retention is a judgement call and a defensible one -- drawing the foul is a skill, and the team keeps the ball. State it in the README. Someone will ask.

VERIFY

The pressured sample is 8 to 12% of all actions. League-wide retention sits between 65 and 80%. Individual players should sit a few points either side of that, not thirty -- a player at 98% or 30% almost always means a volume filter is missing.

CHAPTER 15

# The adjustment

`zone_baselines, player_press_resistance, press_resistance_score`

Everything so far produces a ratio. This chapter turns it into something an analyst would take seriously, and it is the single most valuable idea in the book.

> WHY IT WORKS THIS WAY
>
> A raw retention rate mostly measures **where a player receives the ball**, not how well they handle pressure. A centre-back pressured on his own six-yard line and a winger pressured in the final third are doing entirely different jobs, in entirely different amounts of space, against entirely different defensive intent.
>
> Rank them on the same ratio and you have built a position detector wearing a skill metric's clothes. Deep midfielders will top your table, and they will top it for reasons that have nothing to do with being press-resistant.
>
> The fix is a residual. You already have a grid. Compute the league-average outcome *per zone*, then score every player on actual minus expected given where they received. Now the number means something a ratio never could, and it survives the first hard question anyone asks.

## Building it

1. Bin every pressured action by its start zone, reusing `zone_index` -- the same grid as the xT model, never a second one.
2. Group by zone: mean retention, mean xt_delta. That is `zone_baselines`.
3. Merge those two columns back onto every pressured action by zone.
4. Per player, take the mean of (actual - expected) for both quantities.
5. Apply the minimum-volume filter and carry a position label.

## Two axes, not one number

You were asked for a score. Build it -- but lead your figure with the scatter. Retention above expectation on x, xT above expectation on y. The quadrants name themselves: safe recyclers, risk-takers, the genuinely press-resistant, and the liability.

A single composite hides that structure and forces you to defend arbitrary weights. Keep the weights as function arguments, standardise each axis before combining them -- they are in different units -- and use the score only to sort the table underneath the scatter.

## Optional: shrinkage

A player with 51 pressured actions can post an extreme residual on noise alone. Shrink each estimate toward zero in proportion to sample size:

```
k = 100                                    # tune on the spread you observe
shrunk = residual * n / (n + k)
```

This is empirical-Bayes reasoning in one line: with little evidence, stay near the prior; with plenty, trust the data. It is the same logic behind regression to the mean in every sport, it is three lines, and almost nobody in a portfolio project does it.

TRAP

`test_zone_adjustment_actually_changes_the_ranking` asserts the raw and adjusted rankings correlate at less than 0.99. If they correlate at 1.0, you computed the baselines and never actually subtracted them -- a failure mode that produces a complete, plausible, entirely unadjusted table.

PART IV

# Figures

Ten functions in viz.py. The figures are not decoration on the analysis -- for most readers they are the analysis, and they are what a reviewer looks at before deciding whether to read the code.

CHAPTER 16

# Drawing on a pitch

viz.py

### Let mplsoccer own the coordinates

```
from mplsoccer import Pitch
pitch = Pitch(pitch_type="statsbomb", line_color="#333333")
fig, ax = pitch.draw(figsize=(9, 6))
```

`pitch_type="statsbomb"` configures a 120 by 80 pitch with the origin top-left and y increasing downward. Do not invert y yourself on top of that -- the result looks almost right, which makes it the worst kind of wrong.

### The value surface

```
ax.imshow(model.grid(), extent=[0, 120, 80, 0], aspect="auto",
          cmap="magma", alpha=0.75, zorder=0)
```

Note `extent=[0, 120, 80, 0]` -- the y bounds are reversed, because StatsBomb's origin is top-left while imshow assumes bottom-left. Get this wrong and the pitch is vertically mirrored, which on a near-symmetric surface is genuinely hard to spot. Check against the magnitude table rather than by eye.

### Actions as arrows

```
pitch.arrows(sub.start_x, sub.start_y, sub.end_x, sub.end_y,
             width=2 + 40 * sub.xt_delta, headwidth=4,
             color="#B04C1B", ax=ax)
```

Scale width by value so the eye is drawn to the important actions. Pick a player whose profile is legible -- a deep-lying playmaker and a ball-carrying winger produce strikingly different pictures, and showing both makes the point that xT measures progression *style*, not merely quality.

### The quadrant scatter

Your lead figure. Draw `axhline(0)` and `axvline(0)` so the four quadrants read instantly, label only the extremes, and leave everyone else as unlabelled points. A chart with sixty names on it communicates less than one with six.

### The leaderboard underneath it

The scatter shows structure; the reader still arrives wanting to know who is best. Ranked horizontal bars of the composite score answer that directly, and the two figures belong together -- scatter first, leaderboard immediately below.

Three details do most of the work. Annotate each bar with its sample size, so it is visible that rank three had 340 pressured actions and rank four had 62. Colour by position group with a small legend,

which makes the like-for-like caveat silently rather than in a footnote. And draw a line at zero -- these are residuals above expectation, so zero means 'exactly as expected given where he receives the ball', and that reference point is the entire argument for the adjusted metric.

### The case study

One pressing sequence drawn end to end: pressure locations as markers, the possession that followed as arrows, annotated with the real minute and player names. This is the figure people remember, because it turns a statistical claim into something a coach would recognise.

Remember the pressures need mirroring into the possession frame before they will line up with the possession that followed. If your markers cluster at the opposite end from the arrows, that is the bug.

> VERIFY
>
> Every figure regenerates from a script with no manual steps. Every one is readable in greyscale, because someone will print your README. And every one has a title that states a finding rather than naming a variable -- 'Portugal press high but earn little' beats 'PPDA vs xT'.

PART V

# Appendices

Reference material for while you work.

CHAPTER A

# The idioms

numpy and pandas patterns used in this book

| Idiom | What it does | Where |
|---|---|---|
| `np.bincount(z, minlength=N)` | Count occurrences per integer code, covering every code. | Ch 2, 3 |
| `np.bincount(z, weights=w)` | Grouped sum in one call -- a groupby without the overhead. | Ch 3 |
| `a * n_b + b then bincount` | Contingency table over two integer variables. Fast. | Ch 4 |
| `np.divide(x, y, out=, where=)` | Division with an explicit default. No NaN, no warning. | Ch 2, 3, 4 |
| `m[:, None] * T` | diag(m) @ T without building the diagonal matrix. | Ch 5 |
| `np.linalg.solve(A, b)` | Solve a linear system. Always prefer over inv. | Ch 5 |
| `boolean.cumsum()` | Assign run ids -- the core of any sequence detection. | Ch 11 |
| `np.searchsorted(sorted, v)` | Find insertion points in log time. Forward lookups. | Ch 11 |
| `df.explode(col)` | One row per list element. StatsBomb list columns. | Ch 10 |
| `pd.merge_asof(direction=)` | Nearest-key join. Use 'nearest', not 'forward'. | Ch 10 |
| `residual * n / (n + k)` | Shrink small-sample estimates toward the prior. | Ch 15 |

CHAPTER B

# The thirteen tests

what each one proves

Delete the `@pytest.mark.skip` as you implement. Each test encodes either a measured calibration figure or a specific silent failure.

| Test | Catches | Ch |
|---|---|---|
| `transition_matrix_is_row_stochastic` | Normalising columns instead of rows. | 4 |
| `shot_and_move_probabilities_sum_to_one` | A broken denominator in s. | 2 |
| `iterative_and_closed_form_agree` | An error in either solver. | 5 |
| `grid_has_no_nans` | An unhandled empty zone. | 2, 3 |
| `value_increases_toward_the_goal` | Transposed grid, swapped axes, inverted frame. | 5 |
| `magnitudes_match_the_calibration_table` | Penalties left in g; wrong scale. | 3 |
| `agrees_with_karun_singhs_published_grid` | Any systematic error in the fit. | 6 |
| `frame_mirror_is_an_involution` | A mirror that is not its own inverse. | 9 |
| `ppda_lands_in_the_published_band` | The wrong zone threshold or frame. | 13 |
| `sequences_never_span_a_possession_change` | A break condition missing possession. | 11 |
| `pressured_sample_is_the_right_size` | Keying off Ball Receipts. The 80% loss. | 14 |
| `retention_rate_is_plausible` | A miscoded outcome taxonomy. | 14 |
| `zone_adjustment_actually_changes_the_ranking` | Baselines computed but never subtracted. | 15 |

CHAPTER C

# Calibration targets

every measured number in one place

Measured on Canada-Morocco (WC 2022), Portugal-Czech Republic (Euro 2024) and Morocco-Colombia (WWC 2023). These are targets for your pipeline, not published results -- three matches is a sanity check, not a finding.

| Quantity | Expected |
|---|---|
| Pass completion rate | 0.78 - 0.84 (0.827 on the sample match) |
| Shots per match, both teams | 12 - 24 in the samples; 10 - 30 is normal |
| xT, central zone at the penalty spot | 0.20 - 0.30 |
| xT, edge of the box, central | 0.06 - 0.10 |
| xT, halfway line, central | 0.010 - 0.020 |
| xT, own defensive third | 0.002 - 0.008 |
| Correlation with Karun Singh's grid | Pearson r above 0.97, MAE below 0.01 |
| Team xT vs goals, held out | Spearman rho 0.45 - 0.65 |
| Pressure events per match | 264 - 438 |
| Pressures resolving via related_events | about 93% |
| Pressure to pressured-action gap | -1.2s to +0.9s, two-sided |
| Pressure duration | median 0.47s, max about 3.5s |
| Press to regain, within 8s | about 17.9%, median 3s when it happens |
| High regains as a share of all regains | roughly a third; varies a lot by side |
| Counterpress to regain | about 7.8% -- lower than ordinary pressing |
| PPDA | 7.4 - 12.3 measured; 6 - 15 acceptable |
| Pressing sequences per team per match | 40 - 80 |
| Pressured actions as a share of all actions | 8 - 12% |
| Retention under pressure, league-wide | 65 - 80% |

CHAPTER D

# Where to read more

primary sources

| Source | Why |
|---|---|
| karun.in/blog/expected-threat.html | The primary source for xT. Read twice. |
| nessis.org/nessis11/rudd.pdf | Sarah Rudd, 2011. The original Markov formulation. |
| soccermatics.readthedocs.io -- lesson4/xTPos | The clearest written derivation anywhere. |
| socceraction.readthedocs.io -- valuing_actions/xT | A precise academic statement; your reference implementation. |
| people.cs.kuleuven.be/~pieter.robberechts -- pressing | Valuing the Art of Pressing. Closest prior art to Part II. |
| blogarchive.statsbomb.com -- counter-pressing | How StatsBomb define the pressure event, from the source. |
| mplsoccer.readthedocs.io -- gallery/tutorials/plot_xt | Grid heatmaps and arrows in the shape you need. |
| github.com/statsbomb/open-data -- doc/ | Events v4.0.0 is the authoritative field list. |